



Function Basics in JS

Brief Overview

This note covers **functions in JavaScript** and was created from a [7-page PDF](#). It walks through a playful robot analogy, explains the syntax of defining and calling functions, and compares console.log and alert for debugging.

Key Points

- Robot analogy to illustrate reusable code blocks
 - Function definition: keyword, name, parentheses, body
 - Calling functions and using console.log versus alert for output
-



Robot Analogy for Functions

Function – A named block of code that groups a sequence of instructions, allowing you to execute the same set of steps repeatedly without rewriting them.

- The robot **Angela** receives step-by-step instructions (e.g., alert("leaveHouse"), moveRight).
- Repeating the whole instruction list each time you need milk would be tedious → **DRY** (Don't Repeat Yourself) principle.
- Packaging those steps into a single named block solves the problem – this block is a **function**.



Defining a Function in JavaScript

Syntax Components

- **Keyword**: function – tells the interpreter you're creating a new function.
- **Name**: follows the same rules as variable names (camelCase, cannot start with a number).
- **Parentheses**: () – hold parameters; can be empty for a simple function.
- **Curly Braces**: { ... } – enclose the statements that will run when the function is called.

Example: getMilk Function

```
function getMilk() {  
  // instructions such as console.log('Leave house');  
  // console.log('Move right');  
  // console.log('Move up');  
  // ...  
}
```

- All code **inside** the braces is indented to show it belongs to the function.
- The closing brace **does not** end with a semicolon; the semicolon is used only when **calling** the function.

Naming Rules

Rule	Description
camelCase	First word lower-case, subsequent words capitalized (e.g., getMilk).
No leading numbers	1getMilk is invalid.
Valid characters	Letters, digits, underscores, and dollar signs after the first character.

Style Tips

- Indent the body of the function for readability.
- Place the opening brace on the same line as the function header (common JavaScript style).

Calling a Function

Calling – Executing a function by writing its name followed by parentheses and ending the statement with a semicolon.

- Syntax: getMilk();
- **Only** the function name and parentheses are needed; the function keyword and braces are **not** repeated.
- When called, the interpreter locates the function definition and runs all statements inside its curly braces.

Using console.log vs alert

Purpose Differences

Feature	alert()	console.log()
Visibility	Shown to the user in a pop-up dialog.	Visible only in the browser's developer console.
Use case	Quick user notifications.	Debugging output for developers.
Interaction	Requires user to click OK to continue.	Does not interrupt script execution.

console.log – A developer-only method that prints messages to the console, useful for inspecting values without disturbing the user experience.

Switching from alert to console.log

- Use the find-and-replace feature (Ctrl/Cmd + F) to replace every occurrence of alert with console.log.
- After replacement, running getMilk(); will output the robot's steps in the console instead of showing pop-up alerts.

Summary of Key Points

- Functions encapsulate reusable code, reducing repetition and adhering to the **DRY** principle.
- A JavaScript function definition consists of the function keyword, a valid name, parentheses, and a block delimited by {}.
- Calling a function requires only the name and parentheses, followed by a semicolon.
- console.log is preferred over alert for debugging because it avoids intrusive pop-ups and keeps output in the developer console.